

Análisis cuantitativo avanzado

Clase 3: Regresión Logística

Dr. Lucio José Pantazis

Motivación

Cobertura de salud

La siguiente base “acs_health.Rda” describe datos sobre cobertura de salud en una muestra de estadounidenses residentes en la ciudad de Georgia (disponible en el siguiente link).

El archivo está en el formato “.Rda”, un formato pensado para el lenguaje R. Por lo tanto, debemos usar códigos específicos para descargar y procesar los datos. Podemos utilizar un paquete especial para poder leer este tipo de archivos, llamado rpy2.

En la notebook correspondiente a la clase se puede observar cómo realizar la descarga. No será el foco de esta clase.

Carga de la base e inspección

Ya cargada la base, podemos empezar a inspeccionar sus variables y evaluar su estructura.

```
print(df.head())
```

```
##      id  no.coverage  age  ...      schl      esr
## 0     1             0   60  ...      HS Degree  Employed Civilian
## 1     2             0   44  ... Undergraduate Degree  Employed Civilian
## 2     3             0   40  ... Undergraduate Degree  Employed Civilian
## 3     4             0   76  ...      Graduate Degree  Not in Labor Force
## 4     5             0   71  ...      HS Degree  Not in Labor Force
##
## [5 rows x 11 columns]
```

Inspección de las variables

Además, veamos qué tipo de datos tienen las variables.

```
df.info()
```

```
## <class 'pandas.DataFrame'>
## RangeIndex: 74805 entries, 0 to 74804
## Data columns (total 11 columns):
##  #   Column                Non-Null Count  Dtype
##  ---  ---
##  0    id                    74805 non-null  int64
##  1    no.coverage           74805 non-null  int64
##  2    age                   74805 non-null  int64
##  3    race                  74805 non-null  str
##  4    sex                   74805 non-null  str
##  5    wage                  74805 non-null  float64
##  6    cit                   74805 non-null  str
##  7    mar                   74805 non-null  str
##  8    schl                  74805 non-null  str
##  9    esr                   74805 non-null  str
##  10   puma                  74805 non-null  int64
## dtypes: float64(1), int64(4), str(6)
## memory usage: 6.3 MB
```

- Vemos que la base tiene 74805 registros.
- Entre las variables se encuentran varias ya denominadas como categóricas.

Inspección de las variables

- **id:** Variable identificatoria del individuo (variable cualitativa)
- **no.coverage:** Indica si el individuo tiene cobertura (vale 0) o no tiene cobertura (vale 1) (variable cualitativa)
- **age:** Edad del paciente en años (variable cuantitativa)
- **race:** Categorización del individuo según rasgos étnicos (variable cualitativa)
- **sex:** Sexo biológico del individuo (variable cualitativa)
- **wage:** Ingreso en dólares del individuo (variable cuantitativa)
- **cit:** Variable que indica si el individuo tiene ciudadanía norteamericana (variable cualitativa)
- **mar:** Estado civil del individuo (variable cualitativa)
- **sch1:** Nivel de estudios alcanzados (variable cualitativa ordinal)
- **esr:** Estado laboral del individuo (variable cualitativa)

Variable de respuesta

A partir de esta base, podemos tratar de identificar qué factores influyen sobre tener o no seguro médico. En la base, esta variable se identifica con "no.coverage", que está establecida como numérica (float64) pero en realidad vemos que es una variable dummy, que sólo toma valores 0 y 1 para identificar si un individuo tiene cobertura médica o no. En este caso, se identifica con un 0 si tiene cobertura y con un 1 si no la tiene.

```
print(df["no.coverage"].value_counts())
```

```
## no.coverage
## 0      61114
## 1      13691
## Name: count, dtype: int64
```

Vemos que si bien la mayoría de los individuos de la muestra tienen seguro médico, considerando las dificultades socioeconómicas que introduce en la vida cotidiana, sigue siendo un porcentaje alto el de las personas sin seguro médico (cerca del 18%):

```
print(df["no.coverage"].value_counts()/df.shape[0]*100)
```

```
## no.coverage
## 0      81.697747
## 1      18.302253
## Name: count, dtype: float64
```

Redefinición de la variable

Para simplificar un poco algunos análisis, podemos definir una variable "cobertura" que en vez de ceros y unos, informe efectivamente si tiene cobertura o no:

```
df["cobertura"] = df["no.coverage"].map({0: "Con Cobertura", 1: "Sin Cobertura"})  
print(df["cobertura"].value_counts() / df.shape[0] * 100)
```

```
## cobertura  
## Con Cobertura      81.697747  
## Sin Cobertura      18.302253  
## Name: count, dtype: float64
```

A partir de estos datos, nos puede surgir la siguiente pregunta:

¿Qué variables se vinculan con el hecho de tener o no tener seguro médico?

Análisis exploratorio de datos

Análisis exploratorio de datos

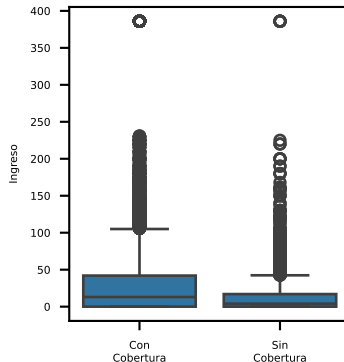
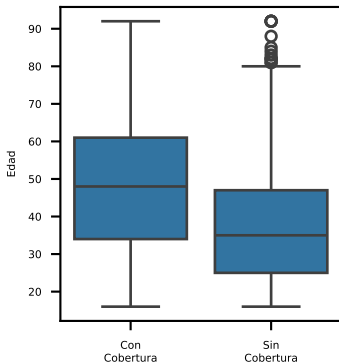
Siempre conviene antes de elegir las variables que consideramos de influencia, ver si hay evidencia que sustente la sospecha. Por lo tanto, vamos a realizar algunos gráficos exploratorios, para lo cual debemos considerar que la variable de respuesta (tener seguro médico o no) es **cualitativa**.

```
import matplotlib.pyplot as plt  
import seaborn as sns
```

Variables cuantitativas (boxplots)

Primero, veremos cómo se vinculan con la cobertura médica las siguientes variables *cuantitativas*: "age" y "wage". La idea es comparar cómo se distribuyen sus valores entre los que tienen seguro médico y no lo tienen. Por lo tanto, podemos comparar sus boxplots

```
fig, axes = plt.subplots(1, 2, figsize=(4, 2.25)); labVars = ["age", "wage"]; titVars = ["Edad", "Ingreso"];
for i in range(2):
    sns.boxplot(data=df, x="cobertura", y=labVars[i], ax=axes[i], fliersize=4)
    axes[i].set_ylabel(titVars[i], fontsize=4); axes[i].set_xlabel(" ", fontsize=1);
    axes[i].set_xticks(ticks=[0, 1], labels=["Con\nCobertura", "Sin\nCobertura"], fontsize=4);
    axes[i].tick_params(axis='both', labelsize=4);
fig.tight_layout(); plt.show()
```



Análisis de boxplots

En ambos casos, se puede ver que si bien los valores mínimos y máximos coinciden entre ambas clases, la distribución es distinta, dando nociones de alguna influencia de ambas variables sobre la variable de cobertura.

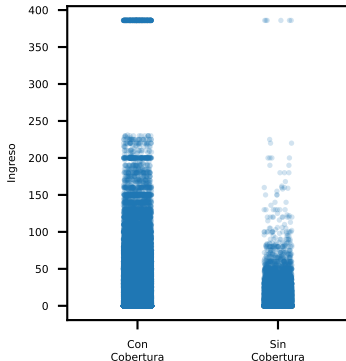
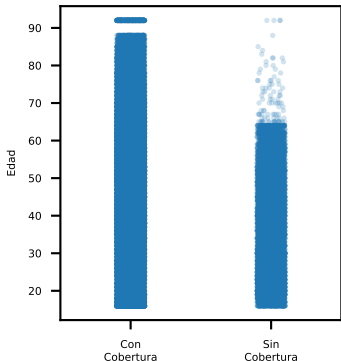
Ambas variables tienden a tener menores valores entre las personas que no tienen cobertura, ya que las medianas y los terceros cuartiles son menores en ambos casos.

- Lo observado tiene cierta lógica para el caso de la variable "wage", ya que las personas que tengan menor salario, es menos probable que puedan acceder a un seguro médico.
- Sin embargo, la intuición nos falla para la variable "age", ya que se podría pensar que a mayor edad, es más costoso conseguir seguro médico. De todas formas, se puede pensar también que el seguro médico se vuelve más necesario a medida que aumenta la edad y es un costo que se afronta a pesar de las dificultades.

Variables cuantitativas (stripplots)

Otra forma de visualizar estas influencias es con un stripplot, que grafica puntos con una cierta dispersión fija que permite ver dónde hay mayor densidad de puntos.

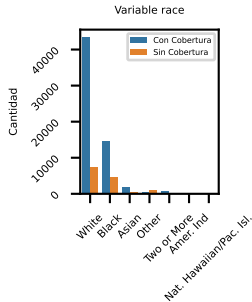
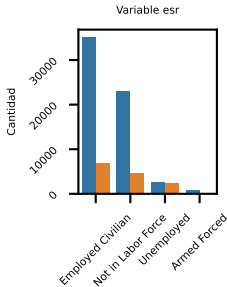
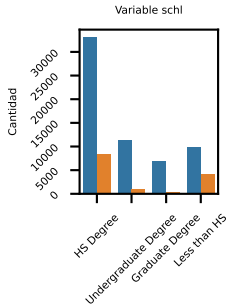
```
fig, axes = plt.subplots(1, 2, figsize=(4, 2.25)); labVars = ["age", "wage"]; titVars = ["Edad", "Ingreso"];
for i in range(2):
    sns.stripplot(data=df, x="cobertura", y=labVars[i], ax=axes[i], alpha=0.2, s=2)
    axes[i].set_ylabel(titVars[i], fontsize=4); axes[i].set_xlabel(" ", fontsize=1);
    axes[i].set_xticks(ticks=[0, 1], labels=["Con\nCobertura", "Sin\nCobertura"], fontsize=4);
    axes[i].tick_params(axis='both', labelsize=4);
fig.tight_layout(); plt.show()
```



Variables cualitativas (countplot)

Ahora vemos cómo se vinculan entre con la cobertura médica las siguientes variables *cualitativas*: "sex", "cit" y "mar". Podemos comparar los resultados en cada clase utilizando un gráfico de barras o countplot:

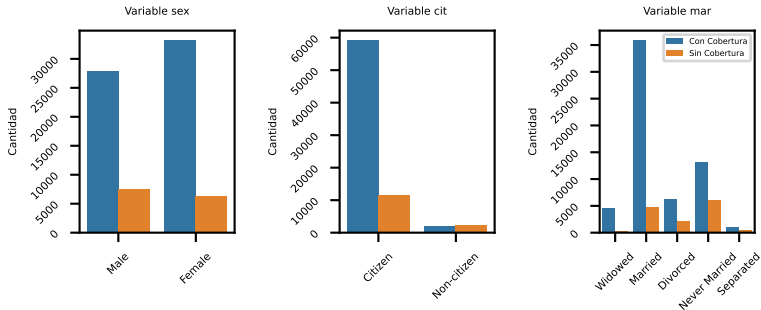
```
labVars=["schl", "esr", "race"]; titVars=["Nivel Educativo", "Situación Laboral", "Etnia"]
fig, axes = plt.subplots(1, 3, figsize=(4.25, 2))
for i in range(0, 3):
    sns.countplot(data=df, x=labVars[i], hue="cobertura", ax=axes[i])
    axes[i].set_title("Variable " + labVars[i], fontsize=4);
    axes[i].set_ylabel("Cantidad", fontsize=4); axes[i].set_xlabel(" ", fontsize=1);
    axes[i].tick_params(axis='both', labelsize=4, labelrotation=45);
    if i < 2:
        axes[i].get_legend().remove();
    else:
        axes[i].legend(fontsize=3);
fig.tight_layout(); plt.show()
```



Variables cualitativas (countplot)

Ahora vemos cómo se vinculan entre con la cobertura médica las siguientes variables *cualitativas*: "schl", "esr" y "race". Podemos comparar los resultados en cada clase utilizando un gráfico de barras o countplot:

```
labVars=["sex", "cit", "mar"]; titVars=["Sexo", "Ciudadanía", "Estado Civil"]
fig, axes = plt.subplots(1, 3, figsize=(4.25, 2))
for i in range(0, 3):
    sns.countplot(data=df, x=labVars[i], hue="cobertura", ax=axes[i])
    axes[i].set_title("Variable " + labVars[i], fontsize=4);
    axes[i].set_ylabel("Cantidad", fontsize=4); axes[i].set_xlabel(" ", fontsize=1);
    axes[i].tick_params(axis='both', labelsize=4, labelrotation=45);
    if i < 2:
        axes[i].get_legend().remove();
    else:
        axes[i].legend(fontsize=3);
fig.tight_layout(); plt.show()
```



Análisis countplots

En general podemos ver que estos gráficos no siempre son muy informativos, sobre todo cuando hay muchas clases que tienen cantidades muy dispares de individuos, ya que no todas las barras se aprecian a la misma escala.

Más aún, tampoco se llega a visualizar lo importante, si la relación en cada clase entre ambas barras es la misma para las personas que tienen cobertura y las personas que no la tienen.

Variables cualitativas (Gráfico de mosaico)

Para evitar estos problemas de visualización, se puede hacer un gráfico de mosaico, que permite comparar lo importante, las **proporciones** en ambas clases.

Para realizar varios gráficos de mosaico, importo la función y agregaremos una función auxiliar definida por nosotros para que los gráficos tengan toda la información pertinente.

Los gráficos de mosaico no se pueden disponer en paneles como los gráficos anteriores de barras, por lo tanto, debemos analizar el vínculo con todas las variables por separado.

La siguiente función realiza un gráfico de mosaico prolijo para vínculos con esta variable binaria:

```
from statsmodels.graphics.mosaicplot import mosaic
# La función ps_mosaic define algunas propiedades del gráfico
def ps_mosaic(df, v1, v2):
    df_c=df[[v1,v2]].value_counts();K=df_c.shape[0];props={};labs={}
    for k in range(K):
        AuxInd=df_c.index[k];labs[AuxInd]=df_c.values[k]
        if AuxInd[1]=="Con Cobertura":
            props[AuxInd]={ 'facecolor': 'red', 'edgecolor': 'white' }
        else:
            props[AuxInd]={ 'facecolor': 'xkcd:aqua', 'edgecolor': 'white' }
    return (props,labs)
```

Gráfico de mosaico (Variable sex)

Ahora procedemos a ver el gráfico de mosaico de la variable "sex" para ver si hay una diferencia por sexo en la cobertura médica:

```
v1="sex";v2="cobertura"
lis=ps_mosaic(df,v1,v2);props=lis[0];labs=lis[1]
labelizer=lambda k:labs[k]
with plt.rc_context({"font.size": 7,"xtick.labelsize": 6,"ytick.labelsize": 6}):
    fig, rects = mosaic(df,[v1,v2], labelizer=labelizer,properties=props,label_rotation=[0,90])
    fig.set_size_inches(2.5,2.5);fig.tight_layout()
    plt.show()
```

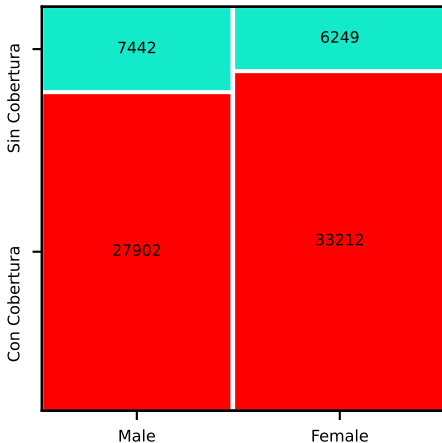


Gráfico de mosaico (Variable cit)

Ahora procedemos a ver el gráfico de mosaico de la variable "cit" para ver si hay una diferencia por ciudadanía en la cobertura médica:

```
v1="cit";v2="cobertura"
lis=ps_mosaic(df,v1,v2);props=lis[0];labs=lis[1]
labelizer=lambda k:labs[k]
with plt.rc_context({"font.size": 7,"xtick.labelsize": 6,"ytick.labelsize": 6}):
    fig, rects = mosaic(df,[v1,v2], labelizer=labelizer,properties=props,label_rotation=[0,90])
    fig.set_size_inches(2.5,2.5);fig.tight_layout()
    plt.show()
```

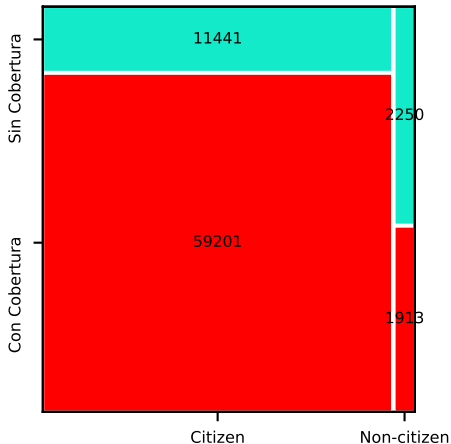


Gráfico de mosaico (Variable mar)

Ahora procedemos a ver el gráfico de mosaico de la variable "mar" para ver si hay una diferencia por estado civil en la cobertura médica:

```
v1="mar";v2="cobertura"
lis=ps_mosaic(df,v1,v2);props=lis[0];labs=lis[1]
labelizer=lambda k:labs[k]
with plt.rc_context({"font.size": 5,"xtick.labelsize": 6,"ytick.labelsize": 6}):
    fig, rects = mosaic(df,[v1,v2], labelizer=labelizer,properties=props,label_rotation=[45,90])
    fig.set_size_inches(2.5,2.5);fig.tight_layout()
    plt.show()
```

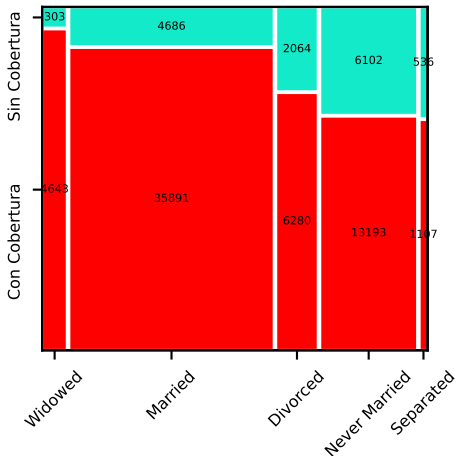


Gráfico de mosaico (Variable sch1)

Ahora procedemos a ver el gráfico de mosaico de la variable "sch1" para ver si hay una diferencia por nivel educativo en la cobertura médica:

```
v1="sch1";v2="cobertura"
lis=ps_mosaic(df,v1,v2);props=lis[0];labs=lis[1]
labelizer=lambda k:labs[k]
with plt.rc_context({"font.size": 5,"xtick.labelsize": 6,"ytick.labelsize": 6}):
    fig, rects = mosaic(df,[v1,v2], labelizer=labelizer,properties=props,label_rotation=[15,90])
    fig.set_size_inches(2.5,2.5);fig.tight_layout()
    plt.show()
```

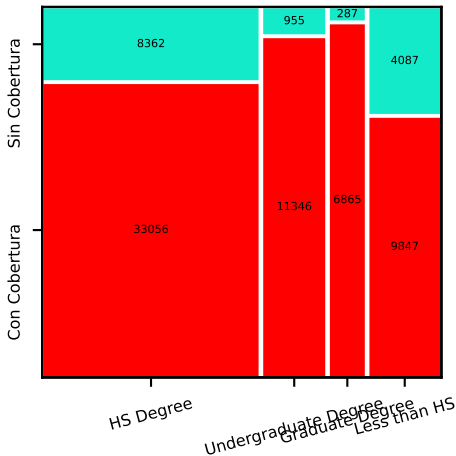


Gráfico de mosaico (Variable esr)

Ahora procedemos a ver el gráfico de mosaico de la variable "esr" para ver si hay una diferencia por situación laboral en la cobertura médica:

```
v1="esr";v2="cobertura"
lis=ps_mosaic(df,v1,v2);props=lis[0];labs=lis[1]
labelizer=lambda k:labs[k]
with plt.rc_context({"font.size": 5,"xtick.labelsize": 6,"ytick.labelsize": 6}):
    fig, rects = mosaic(df,[v1,v2], labelizer=labelizer,properties=props,label_rotation=[15,90])
    fig.set_size_inches(2.5,2.5);fig.tight_layout()
    plt.show()
```

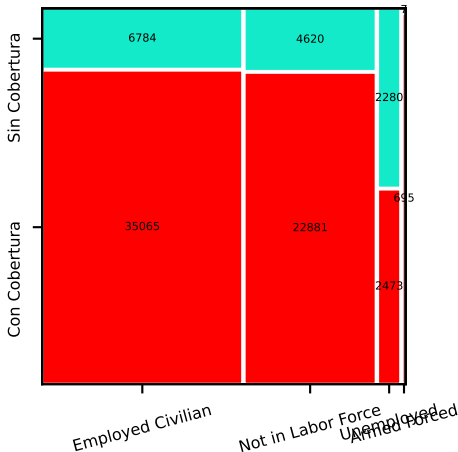
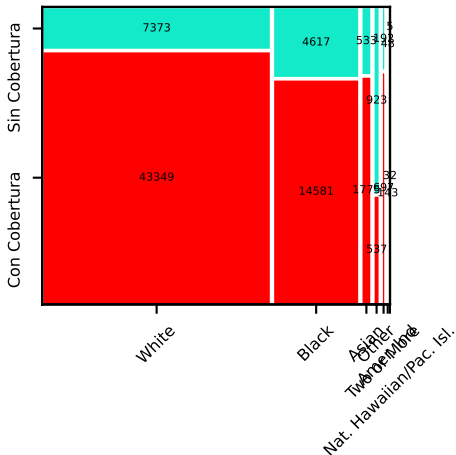


Gráfico de mosaico (Variable race)

Ahora procedemos a ver el gráfico de mosaico de la variable "race" para ver si hay una diferencia por etnia en la cobertura médica:

```
v1="race";v2="cobertura"
lis=ps_mosaic(df,v1,v2);props=lis[0];labs=lis[1]
labelizer=lambda k:labs[k]
with plt.rc_context({"font.size": 5,"xtick.labelsize": 6,"ytick.labelsize": 6}):
    fig, rects = mosaic(df,[v1,v2], labelizer=labelizer,properties=props,label_rotation=[45,90])
    fig.set_size_inches(2.5,2.5);fig.tight_layout()
    plt.show()
```



Análisis Gráficos de Mosaico

Por lo tanto, todas las variables parecen influir sobre la probabilidad de tener cobertura o no, ya que en todos los casos, las proporciones son diferentes por cada clase. De todas formas, esto es un primer indicio, hay que darle sustento desde otro enfoque.

Regresión logística

Regresión logística

Ya vimos en las clases anteriores que los vínculos entre variables se evalúan mediante **modelos de regresión**. Según lo visto, plantear un modelo de regresión lineal para establecer un vínculo entre la variable "cobertura" con las variables "age", "wage", "sex", "cit", "mar", "schl", "esr" y "race" sería plantear el siguiente modelo:

$$\text{cobertura} \approx \beta_0 + \beta_1 \cdot \text{age} + \beta_2 \cdot \text{wage} + \beta_3 \cdot \text{sex} + \beta_4 \cdot \text{cit} + \beta_5 \cdot \text{schl} + \beta_6 \cdot \text{esr} + \beta_7 \cdot \text{race}$$

Regresión logística

Sin embargo, surge un problema ante esta formulación. Más allá de que hay muchas variables predictoras que son categóricas y deben utilizarse las variables “dummy”, la variable “no.coverage” toma sólo los valores 0 y 1.

Por lo tanto, los valores de las constantes $\beta_0, \beta_1, \beta_2, \beta_3, \beta_4, \beta_5, \beta_6, \beta_7$ no son completamente libres, ya que deberían compensarse para que el resultado siempre sea **exactamente** 0 o 1. Esto no tiene mucho sentido, por lo que se necesita una variable de respuesta que tome valores en un continuo.

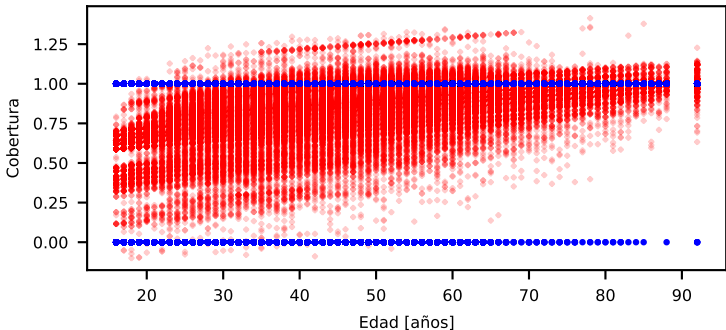
Regresión lineal

Supongamos que queremos realizar una regresión lineal con estas variables (utilizando las variables dummies) y realizamos predicciones para los mismos datos:

```
from sklearn.linear_model import LinearRegression
mod=LinearRegression();mod.fit(X,y);y_pred=mod.predict(X);
```

Para poder evaluar cómo son las predicciones, podemos graficar las mismas junto con las respuestas reales respecto de alguna variable (por ejemplo, la variable de edad):

```
fig=plt.figure(figsize=(4,2))
plt.scatter(X["age"], y_pred,c="r",marker="x",alpha=0.2,s=2);plt.scatter(X["age"],y,c="b",s=2)
plt.xlabel("Edad [años]",fontsize=6);plt.ylabel("Cobertura",fontsize=6);plt.tick_params(axis="both",labelsize=6);
fig.tight_layout();plt.show()
```



Aquí vemos gráficamente que el modelo es erróneo, porque no representa gráficamente a los datos, justamente porque las respuestas deben

Nociones de probabilidad

Supongamos que vamos a tirar una moneda. Intuitivamente, tenemos la idea de que la **probabilidad** de que el resultado sea cara es del 50%. Pero vamos a detenernos a pensar un minuto en qué representa ese valor, para luego poder interpretar el concepto de probabilidad en otros casos.

Una forma de pensar la probabilidad, es pensar en cuán frecuente es algún resultado de un experimento aleatorio. En este caso,

- el experimento es tirar una moneda
- es aleatorio porque no podemos determinar su resultado antes del experimento
- se quiere determinar con qué frecuencia la moneda sale cara

Nociones de probabilidad

El problema es que para determinar cuán frecuente es algo, necesitamos que el experimento se repita varias veces, y nosotros sólo tiramos la moneda una vez. Pero justamente, podemos interpretar el valor de 50% del siguiente modo: si repitiéramos un gran número de veces el experimento (tirar varias veces una moneda), aproximadamente el 50% de esos resultados serían una cara, ya que la moneda no tiene ninguna preferencia por salir de alguno de los dos lados.

```
print(dfEj.iloc[0:24])
```

##	Tirada	Resultado	Caras acumuladas	Porcentaje Caras
## 0	1	ceca	0	0.000000
## 1	2	ceca	0	0.000000
## 2	3	cara	1	33.333333
## 3	4	ceca	1	25.000000
## 4	5	cara	2	40.000000
## 5	6	ceca	2	33.333333
## 6	7	ceca	2	28.571429
## 7	8	cara	3	37.500000
## 8	9	ceca	3	33.333333
## 9	10	ceca	3	30.000000
## 10	11	cara	4	36.363636
## 11	12	ceca	4	33.333333
## 12	13	ceca	4	30.769231
## 13	14	cara	5	35.714286
## 14	15	cara	6	40.000000
## 15	16	ceca	6	37.500000
## 16	17	cara	7	41.176471
## 17	18	cara	8	44.444444
## 18	19	cara	9	47.368421
## 19	20	cara	10	50.000000
## 20	21	cara	11	52.380952
## 21	22	ceca	11	50.000000
## 22	23	cara	12	52.173913
## 23	24	cara	13	54.166667

Nociones de probabilidad

El problema es que para determinar cuán frecuente es algo, necesitamos que el experimento se repita varias veces, y nosotros sólo tiramos la moneda una vez. Pero justamente, podemos interpretar el valor de 50% del siguiente modo: si repitiéramos un gran número de veces el experimento (tirar varias veces una moneda), aproximadamente el 50% de esos resultados serían una cara, ya que la moneda no tiene ninguna preferencia por salir de alguno de los dos lados.

```
print(dfEj.iloc[25:N])
```

	Tirada	Resultado	Caras acumuladas	Porcentaje Caras
## 25	26	ceca	13	50.000000
## 26	27	cara	14	51.851852
## 27	28	cara	15	53.571429
## 28	29	ceca	15	51.724138
## 29	30	cara	16	53.333333
## 30	31	cara	17	54.838710
## 31	32	ceca	17	53.125000
## 32	33	cara	18	54.545455
## 33	34	cara	19	55.882353
## 34	35	cara	20	57.142857
## 35	36	ceca	20	55.555556
## 36	37	ceca	20	54.054054
## 37	38	cara	21	55.263158
## 38	39	ceca	21	53.846154
## 39	40	ceca	21	52.500000
## 40	41	ceca	21	51.219512
## 41	42	ceca	21	50.000000
## 42	43	ceca	21	48.837209
## 43	44	cara	22	50.000000
## 44	45	ceca	22	48.888889
## 45	46	cara	23	50.000000
## 46	47	cara	24	51.063830
## 47	48	cara	25	52.083333
## 48	49	cara	26	53.061224

Probabilidad como respuesta

Por lo tanto, vamos a pensar en la **probabilidad** de que una persona tenga cobertura médica. Es decir, si tuviéramos un gran número de personas **con las mismas características**, ¿qué proporción de ellas tendría cobertura médica?

Por lo tanto, vamos a adaptar el modelo anterior del siguiente modo:

$$P(\text{cobertura} = 1) \approx \beta_0 + \beta_1 \cdot \text{age} + \beta_2 \cdot \text{wage} + \beta_3 \cdot \text{sex} + \beta_4 \cdot \text{cit} + \beta_5 \cdot \text{schl} + \beta_6 \cdot \text{esr} + \beta_7 \cdot \text{race}$$

donde P representa la probabilidad de que la variable "cobertura" sea 1. La ventaja de esta propuesta, es que el valor de la probabilidad ya no tiene porqué valer exactamente 0 o 1, sino que puede tomar varios valores. Sin embargo, siguen siendo valores que tienen sentido **entre 0 y 1**. Por lo tanto, los valores de cada constante β siguen estando restringidos.

Odds

Para ampliar el rango de valores posibles, se puede comparar la probabilidad de que una persona tenga cobertura, respecto de la probabilidad de que no la tenga. Para eso se define el siguiente cociente denominado “odds”:

$$\text{odds} = \frac{P(\text{cobertura} = 1)}{P(\text{cobertura} = 0)}$$

Considerando que la variable tiene sólo dos valores, en realidad, se puede expresar del siguiente modo:

$$P(\text{cobertura} = 0) = 1 - P(\text{cobertura} = 1)$$

Entonces, podemos reescribir el odds del siguiente modo:

$$\text{odds} = \frac{P(\text{cobertura} = 1)}{1 - P(\text{cobertura} = 1)}$$

- A partir de esta división, si el resultado es mayor que 1, significa que es más probable que una persona tenga cobertura, respecto de que no la tenga. Más aún, si el resultado del cociente es 8, significa que la probabilidad de que una persona tenga cobertura es 8 veces más grande que la probabilidad de que no la tenga.
- Del mismo modo, si el resultado es menor que 1, significa que es menos probable que una persona tenga cobertura, respecto de las personas sin seguro médico.
- El caso en el que el odds vale 1, la probabilidad de cobertura es de $\frac{1}{2} = 0.5$.

Odds como respuesta

Por lo tanto, podemos actualizar el modelo del siguiente modo:

$$\text{odds} = \frac{P(\text{cobertura} = 1)}{1 - P(\text{cobertura} = 1)} \approx \beta_0 + \beta_1 \cdot \text{age} + \beta_2 \cdot \text{wage} + \beta_3 \cdot \text{sex} + \beta_4 \cdot \text{cit} + \beta_5 \cdot \text{schl} + \beta_6 \cdot \text{esr} + \beta_7 \cdot \text{race}$$

Respecto a los valores que puede tomar el odds,

- si el valor de $P(\text{cobertura} = 1)$ es cercano a 0, el denominador es cercano a 1 y el resultado del odds es cercano a 0
- si el valor de $P(\text{cobertura} = 1)$ es cercano a 1, el denominador es cercano a 0 y el resultado del odds puede tomar valores muy altos, abarcando cualquier número positivo.

De todas formas, el resultado del odds siempre es **positivo**. Por lo tanto, sigue habiendo una restricción para las constantes β .

Log Odds como respuesta

Para resolver todo tipo de restricción, se puede tomar el logaritmo natural al cociente odds, ya que al aplicarlo a cualquier número positivo, el logaritmo puede tomar cualquier valor real, ya sea positivo o negativo.

Entonces, el modelo final que se plantea es el siguiente:

$$\log(\text{odds}) = \log\left(\frac{P(\text{cobertura} = 1)}{1 - P(\text{cobertura} = 1)}\right) \approx \beta_0 + \beta_1 \cdot \text{age} + \beta_2 \cdot \text{wage} + \beta_3 \cdot \text{sex} + \beta_4 \cdot \text{cit} + \beta_5 \cdot \text{schl} + \beta_6 \cdot \text{esr} + \beta_7 \cdot \text{race}$$

En este caso, como el logaritmo puede tomar cualquier valor real, las constantes β ya no tienen ningún tipo de restricción. Por lo tanto, buscamos que los coeficientes estimados se atengan a este modelo. La función $\log(\text{odds}(p))$ es a veces llamada “logit”.

Visualización numérica

El vínculo entre las probabilidades p , el odds y el log odds, lo podemos ver dándole valores a p :

```
ps=np.linspace(0.1,0.9,9)
odds=ps/(1-ps)
lOdds=np.log(odds)
df_odds=pd.DataFrame({"p":ps,"odds":odds,"lOdds":lOdds})
print(df_odds)
```

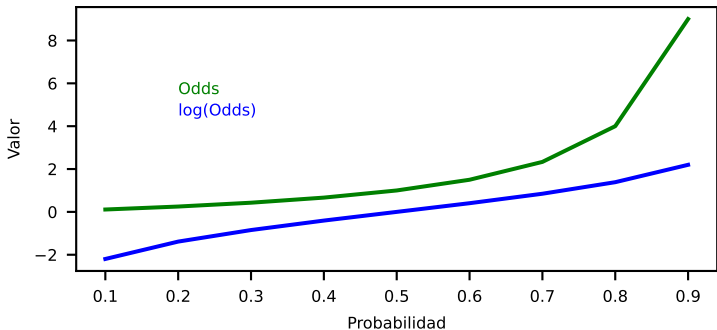
##	p	odds	lOdds
## 0	0.1	0.111111	-2.197225
## 1	0.2	0.250000	-1.386294
## 2	0.3	0.428571	-0.847298
## 3	0.4	0.666667	-0.405465
## 4	0.5	1.000000	0.000000
## 5	0.6	1.500000	0.405465
## 6	0.7	2.333333	0.847298
## 7	0.8	4.000000	1.386294
## 8	0.9	9.000000	2.197225

Vemos que a partir de probabilidades entre 0 y 1, se obtienen valores positivos para el odds, y de ambos signos para el log Odds.

Explicación gráfica

Gráficamente, graficando el odds con color verde y el log Odds con color azul:

```
fig=plt.figure(figsize=(4,2))
plt.plot(ps,odds,c="g");plt.plot(ps,lOdds,c="b");
plt.text(x=0.2,y=5.5,s="Odds",c="g",size=6);
plt.text(x=0.2,y=4.5,s="log(Odds)",c="b",size=6);
plt.xlabel("Probabilidad",fontsize=6);plt.ylabel("Valor",fontsize=6);
plt.tick_params(axis="both",labelsize=6);fig.tight_layout()
plt.show()
```



Estimación

Ahora que ya tenemos el modelo, debemos encontrar los valores óptimos para los coeficientes $\beta_0, \beta_1, \beta_2, \beta_3, \beta_4, \beta_5, \beta_6$ y β_7 . Estos valores deben buscarse con algún criterio razonable. En este caso, a diferencia de lo observado en regresión lineal, se apela a los “estimadores de máxima verosimilitud”.

La idea es la siguiente: Se tiene como dato tanto la cobertura médica o no de cada individuo, y de las variables que influyen sobre dicha condición. A su vez, según el modelo, cada variable tiene una cierta influencia sobre la probabilidad de cobertura (a través de cada β).

¿Cuáles son los valores de β que, a partir de las variables explicativas, aumentan la probabilidad de tener cobertura médica entre los que la tienen y, al mismo tiempo, aumentan la probabilidad de no tener cobertura médica entre los que no la tienen?

Es decir, dados los datos de cobertura y las variables explicativas, ¿cuáles son los valores más verosímiles para cada β ?

Estimación en Python

Para estimar los valores de β , en Python se utiliza la función `LogisticRegression` del paquete `sklearn`. Recordemos que ya fueron generadas las variables dummy. La variable de respuesta fue definida como y , mientras que las covariables fueron definidas como X .

Ahora que tenemos los datos correspondientes para la regresión logística, calculamos los valores de los β con la función `LogisticRegression`:

```
from sklearn.linear_model import LogisticRegression
logreg = LogisticRegression(fit_intercept=False); logreg.fit(X,y)
```

```
## LogisticRegression(fit_intercept=False)

dic={}
for key,val in list(zip(covs,logreg.coef_[0])):
    dic[key]=val
df_coef=pd.DataFrame(dic,index=["Coeficiente"])
print(df_coef.T.head(10))
```

##	Coeficiente
## age	0.030452
## wage	0.026547
## race_Amer. Ind	-0.023452
## race_Asian	0.167211
## race_Black	-0.165208
## race_Nat. Hawaiian/Pac. Isl.	0.014835
## race_Other	-0.777557
## race_Two or More	0.121957
## race_White	0.170344
## sex_Female	-0.019140

Interpretación de los coeficientes

Analizaremos el primer coeficiente, que corresponde a la variable de edad ($\beta_1=0.03045$). La interpretación inicial es similar a la de la regresión múltiple, sólo que debe adaptarse al modelo elegido, cuya variable de respuesta es el log(odds).

Es decir, **manteniendo todas las otras variables constantes**, cada año extra que vive una persona *aumenta* (el signo es positivo) en 0.03045 el valor estimado del log(odds) de tener cobertura médica.

Para independizarse del logaritmo, se aplica una función exponencial, utilizando el número irracional $e \approx 2.71$. Por lo tanto, **manteniendo todas las otras variables constantes**, cada año extra que vive una persona *aumenta* en $e^{0.03045} \approx 1.03092$ el valor estimado del odds de tener cobertura médica, que se calcula del siguiente modo:

$$\frac{P(\text{cobertura} = 1)}{1 - P(\text{cobertura} = 1)}$$

Interpretación de los coeficientes

Sigue sin ser muy interpretable, pero despejando, se puede demostrar que el impacto estimado de cada año vivido sobre la probabilidad de tener seguro médico (manteniendo todas las otras variables constantes) viene dada por la siguiente fórmula

$$\frac{e^{\beta_1}}{1 + e^{\beta_1}} \approx \frac{e^{0.03045}}{1 + e^{0.03045}} \approx 0.5076$$

De todas formas, se puede hacer una interpretación más rudimentaria, en la que se pueden asociar los signos negativos de los coeficientes con una reducción en la probabilidad de tener cobertura médica, mientras que se da lo inverso para coeficientes positivos. A medida que estos coeficientes se alejan del cero, tiene más influencia sobre la probabilidad estimada de tener cobertura médica.

Predicción de probabilidad

En python, una vez estimados los coeficientes β , se puede estimar la probabilidad de tener cobertura sabiendo los valores de las variables predictivas.

Rápidamente, en python se puede hacer del siguiente modo, estimando por ejemplo, la probabilidad de el primer individuo (fila 0) tenga cobertura médica:

```
print(logreg.predict_proba(X)[0,1])
```

```
## 0.9680035456834255
```

Recordemos cómo se interpreta esta probabilidad. El hecho de que dé como resultado 96.8% significa que, según el modelo, si se tuvieran un gran número de personas con **las mismas características** que ese individuo, aproximadamente el 96.8% de ellas tendría cobertura médica.

Verificación del cálculo

Para ver cómo se obtiene este valor, explicaremos a continuación cómo se puede estimar la probabilidad para el primer individuo. Primero vemos los valores de las variables para este individuo y los coeficientes correspondientes

```
print(X.iloc[[0],:])
```

```
##      age  wage  ...  esr_Not in Labor Force  esr_Unemployed
## 0    60  66.0  ...                      False              False
##
## [1 rows x 26 columns]
```

```
print(df_coef)
```

```
##              age      wage  ...  esr_Not in Labor Force  esr_Unemployed
## Coeficiente  0.030452  0.026547  ...                0.22679      -1.047915
##
## [1 rows x 26 columns]
```

Multiplicando cada valor con su respectivo β se obtiene lo siguiente:

```
dfProd=X.iloc[[0],:]*df_coef.values
print(dfProd)
```

```
##              age      wage  ...  esr_Not in Labor Force  esr_Unemployed
## 0  1.827092  1.752105  ...                0.0              -0.0
##
## [1 rows x 26 columns]
```

Vemos que multiplicando los primeros dos β con el valor de la variable correspondiente se obtienen los valores del producto:

- $60 * 0.0304515 = 1.8270916$
- $66 * 0.026547 = 1.7521051$

Verificación del cálculo

Entonces, sumando todos estos valores se obtiene la estimación del $\log(\text{odds})$:

```
expo=np.sum(dfProd.values)
print(expo)
```

```
## 3.4096106561123496
```

Recordando cómo se obtiene la probabilidad a partir del $\log(\text{odds})$, se termina obteniendo el valor estimado de la probabilidad del siguiente modo:

$$\hat{p} = \frac{e^{\log(\text{odds})}}{1 + e^{\log(\text{odds})}}$$

.

Vemos que con esta cuenta da como resultado el mismo que la predicción dada por python.

```
print(np.exp(expo)/(1+np.exp(expo)))
```

```
## 0.9680035456834254
```

Predicciones de probabilidad

Ahora, en el siguiente vector, vamos a guardar las probabilidades estimadas para todos los individuos. Aquí mostramos las probabilidades estimadas para los primeros 5:

```
y_pred_probs = logreg.predict_proba(X)[: , 1]  
print(y_pred_probs[range(5)])
```

```
## [0.96800355 0.94459368 0.94796374 0.97877359 0.94924153]
```

Predicción de respuesta

De todas formas, en la práctica, estas probabilidades estimadas pueden servir para hacer un análisis fino de la cobertura médica, pero en definitiva, para nuevas observaciones queremos saber más que nada si para una nueva observación, predecimos si va a tener cobertura medica o no. Es decir, es de interés clasificar a cada individuo según la primer variable binaria que utilizamos como variable de respuesta.

Para clasificar los nuevos individuos según si tienen cobertura o no, el criterio por default para la regresión logística sería clasificarlos como personas con cobertura médica si la probabilidad estimada es mayor que 50% y clasificarla como sin cobertura en caso contrario. Eso en python se hace con el método `predict`:

```
y_pred=logreg.predict(X)  
print(y_pred_probs[range(5)])
```

```
## [0.96800355 0.94459368 0.94796374 0.97877359 0.94924153]
```

```
print(y_pred[range(5)])
```

```
## [1 1 1 1 1]
```

Notemos que todas las respuestas predichas son 1, es decir, se predice que esas 5 personas tienen cobertura médica. Esto es consistente con que todas las probabilidades estimadas eran mayores que 50%. Más aún, dan cercanos a 95%.

Predicción de respuesta

Podemos ver que los individuos clasificados como 1 tienen probabilidad estimada mayor que 0.5 y los clasificados como 0 tienen probabilidad estimada por debajo de 0.5:

```
## Individuos estimados con cobertura médica:
```

```
## Probabilidad Estimada
```

```
## [0.96800355 0.94459368 0.94796374 ... 0.99908615 0.88394585 0.97379619]
```

```
## Respuesta Estimada
```

```
## [1 1 1 ... 1 1 1]
```

```
## Individuos estimados sin cobertura médica:
```

```
## Probabilidad Estimada
```

```
## [0.44596623 0.4219707 0.37581242 ... 0.42400597 0.45114774 0.39952503]
```

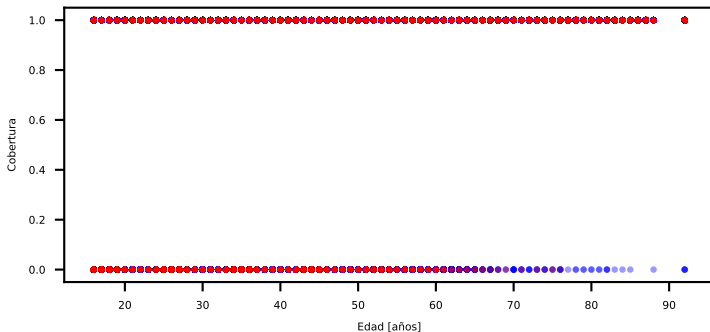
```
## Respuesta Estimada
```

```
## [0 0 0 ... 0 0 0]
```

Verificación gráfica

Ahora que se pueden predecir las respuestas binarias, repetimos el gráfico en el que se superponen los valores reales con sus predicciones correspondientes para ver que el modelo se adapta mejor a los datos que la regresión lineal:

```
fig=plt.figure(figsize=(4,2))
plt.scatter(X["age"],y,alpha=0.4,c="b",s=2)
plt.scatter(X["age"],y_pred,alpha=0.2,c="r",marker="x",s=2)
plt.xlabel("Edad [años]",fontsize=4);plt.ylabel("Cobertura",fontsize=4)
plt.tick_params(axis="both",labelsize=4)
fig.tight_layout();plt.show()
```



Evaluación de las respuestas

Estas predicciones pueden tener errores, y en este caso sabemos las respuestas reales. Para evaluar la calidad de estas predicciones, podemos generar lo que se llama una matriz de confusión.

```
from sklearn.metrics import confusion_matrix
CMat=confusion_matrix(y, y_pred)
dfCM=pd.DataFrame({"Clas s/SM":CMat[:,0], "Clas c/SM":CMat[:,1]}, index=["s/SM", "c/SM"])
print(dfCM)
```

##	Clas s/SM	Clas c/SM
## s/SM	3135	10556
## c/SM	1947	59167

Evaluación de las respuestas

De todas las predicciones, las que son correctas están sobre la diagonal, ya que coincide la clase real con la estimada. Es decir, si consideramos que una persona con seguro médico es una detección “positiva”, tendríamos:

- $VN = 3135$ (verdaderos “negativos”)
- $FN = 10556$ (falsos “negativos”)
- $FP = 1947$ (falsos “positivos”)
- $VN = 59167$ (verdaderos “positivos”)

Esto da lugar a una primer métrica de calidad de clasificación denominada “accuracy” que cuenta la proporción de clasificaciones correctas:

$$\text{Acc} = \frac{VP + VN}{VP + FN + FP + VN} = \frac{3135 + 59167.00}{3135 + 10556.00 + 1947 + 59167.00} \approx 0.8329$$

Es decir, bajo estas estimaciones, se predican correctamente el 83.29% de los datos.

En python se calcula del siguiente modo:

```
print(logreg.score(X,y))
```

```
## 0.832858766125259
```

Evaluación de las predicciones

Sin embargo, pueden haber métricas más complicadas, que evalúan si hay tendencias de clasificaciones erróneas en algunas clases particulares.

- Por ejemplo, notemos que entre los clasificados como “negativos”, es menor la proporción de verdaderos negativos ($\frac{\text{Verdaderos negativos}}{\text{Clasificados Negativos}} = \frac{3135}{1947+3135} \approx 0.6169$) que la proporción de verdaderos positivos entre los clasificados como “positivos” ($\frac{\text{Verdaderos positivos}}{\text{Clasificados Positivos}} = \frac{59167}{10556+59167} \approx 0.8486$).
- Del mismo modo, entre los realmente “negativos”, las clasificaciones correctas son una proporción menor ($\frac{\text{Verdaderos negativos}}{\text{Negativos totales}} = \frac{3135}{10556+3135} \approx 0.229$), respecto de las clasificaciones correctas entre los realmente “positivos” ($\frac{\text{Verdaderos positivos}}{\text{Positivos totales}} = \frac{0.0319}{1947+59167} \approx 0.9681$).

Estas magnitudes van mostrando que los errores al predecir las respuestas de los individuos sin cobertura médica son mayores que los errores al predecir los errores de los individuos con cobertura médica. Es decir, la regresión logística propuesta predice mejor a la clase mayoritaria de las personas con seguro médico. El valor del “accuracy” no permitía ver estos sesgos.

Evaluación de las predicciones

Estas cuestiones se pueden ver con la función `classification_report`.

```
from sklearn.metrics import classification_report
print(classification_report(y, y_pred))
```

##		precision	recall	f1-score	support
##					
##	0	0.62	0.23	0.33	13691
##	1	0.85	0.97	0.90	61114
##					
##	accuracy			0.83	74805
##	macro avg	0.73	0.60	0.62	74805
##	weighted avg	0.81	0.83	0.80	74805

Interpretando estos valores,

- la métrica “precision” representa cuantas de las clasificaciones de cada clase son correctas.
- la métrica “recall” representa la proporción de cada clase que fueron correctamente clasificadas.
- la métrica “f1-score” combina de forma equitativa tanto las métricas “precision” y “recall”, aunque no es exactamente un promedio entre ambas.

Curva ROC

Curva ROC

Otra forma de evaluar la calidad de un clasificador **binario** es utilizando lo que se llama la curva ROC (Receiver Operating Characteristic), en el que se evalúan en simultáneo los falsos y los verdaderos positivos. Los primeros se ubican en la coordenada x y los segundos en la coordenada y .

Hemos visto que según la clasificación como “positiva” (con valor 1) si la probabilidad estimada era mayor que 0.5 y “negativa” (con valor 0) si la probabilidad estimada era menor que 0.5, obtuvimos una proporción de falsos positivos entre los realmente

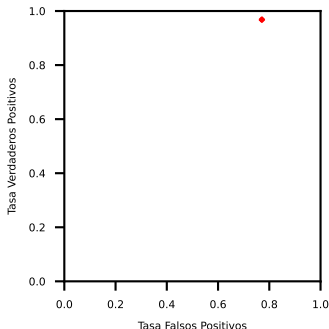
negativos dada por $x = \frac{10556}{3135 + 10556} \approx 0.771$ mientras que entre los positivos, la

proporción de verdaderos positivos es $y = \frac{59167}{1947 + 59167} \approx 0.9681$.

Curva ROC

En el plano ROC, se grafica el punto del siguiente modo:

```
x1=CMat[0,1]/(CMat[0,1]+CMat[0,0]);y1=CMat[1,1]/(CMat[1,0]+CMat[1,1])  
fig=plt.figure(figsize=(2,2));plt.scatter(x1,y1,c="r",marker="x",s=2)  
plt.xlim([0,1]);plt.ylim([0,1]);plt.tick_params(axis="both",labelsize=4);  
plt.xlabel('Tasa Falsos Positivos',fontsize=4);plt.ylabel('Tasa Verdaderos Positivos',  
fig.tight_layout();plt.show()
```



Curva ROC

De todas formas, una vez estimadas las probabilidades, se pueden establecer distintos umbrales (en vez de 0.5) a partir de los cuales, las probabilidades estimadas por debajo de dicho valor, se clasifiquen como 0 y en caso contrario, se clasifiquen como 1. Ahora mostraremos tomando como umbrales 0.1, 0.25, 0.75, 0.9.

```
ps=[0.1,0.25,0.75,0.9];xs=[0]*4;ys=[0]*4;lcm=[]  
for i in range(4):  
    y_pred_Aux=(y_pred_probs>=ps[i]) * 1  
    CMat=confusion_matrix(y, y_pred_Aux)  
    xs[i]=CMat[0,1]/np.sum(CMat[0,:])  
    ys[i]=CMat[1,1]/np.sum(CMat[1,:])  
    lcm=lcm+[pd.DataFrame({"Clas s/SM":CMat[:,0],"Clas c/SM":CMat[:,1]},index=["s/SM","c/SM"])]
```

Con umbral $p=0.1$:

```
print(lcm[0])
```

```
##          Clas s/SM  Clas c/SM  
## s/SM          120      13571  
## c/SM           16      61098
```

```
print("Tasa Falsos Positivos:"+str(xs[0]))
```

```
## Tasa Falsos Positivos:0.9912351179607041
```

```
print("Tasa Verdaderos Positivos:"+str(ys[0]))
```

```
## Tasa Verdaderos Positivos:0.9997381941944563
```


Curva ROC

De todas formas, una vez estimadas las probabilidades, se pueden establecer distintos umbrales (en vez de 0.5) a partir de los cuales, las probabilidades estimadas por debajo de dicho valor, se clasifiquen como 0 y en caso contrario, se clasifiquen como 1. Ahora mostraremos tomando como umbrales 0.1, 0.25, 0.75, 0.9.

```
ps=[0.1,0.25,0.75,0.9];xs=[0]*4;ys=[0]*4;lcm=[]  
for i in range(4):  
    y_pred_Aux=(y_pred_probs>=ps[i]) * 1  
    CMat=confusion_matrix(y, y_pred_Aux)  
    xs[i]=CMat[0,1]/np.sum(CMat[0,:])  
    ys[i]=CMat[1,1]/np.sum(CMat[1,:])  
    lcm=lcm+[pd.DataFrame({"Clas s/SM":CMat[:,0],"Clas c/SM":CMat[:,1]},index=["s/SM","c/SM"])]
```

Con umbral $p=0.25$:

```
print(lcm[1])
```

```
##          Clas s/SM  Clas c/SM  
## s/SM          830      12861  
## c/SM          235      60879
```

```
print("Tasa Falsos Positivos:"+str(xs[1]))
```

```
## Tasa Falsos Positivos:0.9393762325615368
```

```
print("Tasa Verdaderos Positivos:"+str(ys[1]))
```

```
## Tasa Verdaderos Positivos:0.9961547272310763
```

Curva ROC

De todas formas, una vez estimadas las probabilidades, se pueden establecer distintos umbrales (en vez de 0.5) a partir de los cuales, las probabilidades estimadas por debajo de dicho valor, se clasifiquen como 0 y en caso contrario, se clasifiquen como 1. Ahora mostraremos tomando como umbrales 0.1, 0.25, 0.75, 0.9.

```
ps=[0.1,0.25,0.75,0.9];xs=[0]*4;ys=[0]*4;lcm=[]  
for i in range(4):  
    y_pred_Aux=(y_pred_probs>=ps[i]) * 1  
    CMat=confusion_matrix(y, y_pred_Aux)  
    xs[i]=CMat[0,1]/np.sum(CMat[0,:])  
    ys[i]=CMat[1,1]/np.sum(CMat[1,:])  
    lcm=lcm+[pd.DataFrame({"Clas s/SM":CMat[:,0],"Clas c/SM":CMat[:,1]},index=["s/SM","c/SM"])]
```

Con umbral $p=0.75$:

```
print(lcm[2])
```

```
##          Clas s/SM  Clas c/SM  
## s/SM          8667      5024  
## c/SM          11856     49258
```

```
print("Tasa Falsos Positivos:"+str(xs[2]))
```

```
## Tasa Falsos Positivos:0.3669563947118545
```

```
print("Tasa Verdaderos Positivos:"+str(ys[2]))
```

```
## Tasa Verdaderos Positivos:0.8060018980920902
```

Curva ROC

De todas formas, una vez estimadas las probabilidades, se pueden establecer distintos umbrales (en vez de 0.5) a partir de los cuales, las probabilidades estimadas por debajo de dicho valor, se clasifiquen como 0 y en caso contrario, se clasifiquen como 1. Ahora mostraremos tomando como umbrales 0.1, 0.25, 0.75, 0.9.

```
ps=[0.1,0.25,0.75,0.9];xs=[0]*4;ys=[0]*4;lcm=[]  
for i in range(4):  
    y_pred_Aux=(y_pred_probs>=ps[i]) * 1  
    CMat=confusion_matrix(y, y_pred_Aux)  
    xs[i]=CMat[0,1]/np.sum(CMat[0,:])  
    ys[i]=CMat[1,1]/np.sum(CMat[1,:])  
    lcm=lcm+[pd.DataFrame({"Clas s/SM":CMat[:,0],"Clas c/SM":CMat[:,1]},index=["s/SM","c/SM"])]
```

Con umbral $p=0.9$:

```
print(lcm[3])
```

```
##          Clas s/SM  Clas c/SM  
## s/SM          12506          1185  
## c/SM          28174          32940
```

```
print("Tasa Falsos Positivos:"+str(xs[3]))
```

```
## Tasa Falsos Positivos:0.0865532101380469
```

```
print("Tasa Verdaderos Positivos:"+str(ys[3]))
```

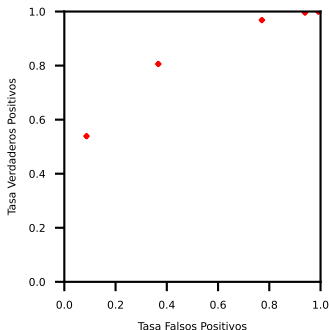
```
## Tasa Verdaderos Positivos:0.5389927021631705
```

Curva ROC

Vemos que menores umbrales dan mayor número de detecciones “positivas” y por lo tanto, mayor proporción de verdaderos positivos pero a costa de mayores falsos positivos. Sumemos al gráfico estos puntos:

```

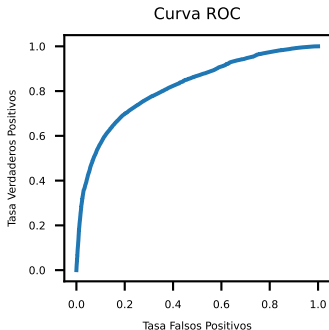
fig=plt.figure(figsize=(2,2));
plt.scatter(x1,y1,c="r",marker="x",s=2);plt.scatter(xs,ys,c="r",marker="x",s=2)
plt.xlim([0,1]);plt.ylim([0,1]);plt.tick_params(axis="both",labelsize=4);
plt.xlabel('Tasa Falsos Positivos',fontsize=4);plt.ylabel('Tasa Verdaderos Positivos',
fig.tight_layout();plt.show()
  
```



Curva ROC

Dando muchos más valores a los umbrales, recorriendo valores entre 0 y 1, se pueden unir con una línea y obtener la curva que se denomina "Curva ROC":

```
from sklearn.metrics import roc_curve
fpr, tpr, thresholds = roc_curve(y, y_pred_probs)
fig=plt.figure(figsize=(2,2));plt.plot(fpr, tpr)
plt.xlabel('False Positive Rate');plt.ylabel('True Positive Rate')
plt.title('Curva ROC',fontsize=6);plt.tick_params(axis="both",labelsize=4);
plt.xlabel('Tasa Falsos Positivos',fontsize=4);plt.ylabel('Tasa Verdaderos Positivos',fontsize=4)
fig.tight_layout();plt.show()
```



Área bajo la curva ROC

Obviamente no tenemos detecciones perfectas, pero si pudiéramos tenerlas al menos como objetivo, tendríamos un 0% de falsos positivos y un 100% de verdaderos positivos. Es decir, la detección perfecta está en el punto (0,1) de ese espacio (la esquina superior izquierda). Por lo tanto, mientras más se acerquen la combinaciones de falsos positivos y verdaderos positivos a ese punto, mejor serán las clasificaciones. Por lo tanto, muchas veces se suele calcular el área bajo esa curva, para evaluar cuánto se acerca la curva a ese valor óptimo.

```
from sklearn.metrics import roc_auc_score  
print(roc_auc_score(y, y_pred_probs))
```

```
## 0.8174189879393283
```

Esta área tiene un valor máximo de 1 (área del cuadrado que limita ambas coordenadas), mientras que las áreas debajo de 0.5 (correspondiente al triángulo bajo la diagonal) dan muestras de malas clasificaciones, ya que tendría menor rendimiento que asignar al azar 50% de las observaciones a una clase y al otro 50% a la clase restante.

Comparación con otro modelo

Supongamos que planteamos un peor modelo, en el que el hecho de tener cobertura médica depende **únicamente** de la edad de la persona. Por lo que hemos visto, hay muchos más factores que influyen sobre la cobertura médica, por lo que deberían arrojar peores métricas de evaluación:

```
# Considero en este modelo simple sólo la variable "age"  
X0=df[["age"]]  
logreg0 = LogisticRegression(fit_intercept=False);logreg0.fit(X0,y);  
y_pred0=logreg0.predict(X0);y_pred_probs0=logreg0.predict_proba(X0)[: ,1]
```

Veamos cuánto da la métrica “accuracy”:

```
print("Accuracy modelo completo: "+str(logreg.score(X,y)))
```

```
## Accuracy modelo completo: 0.832858766125259
```

```
print("Accuracy modelo simple: "+str(logreg0.score(X0,y)))
```

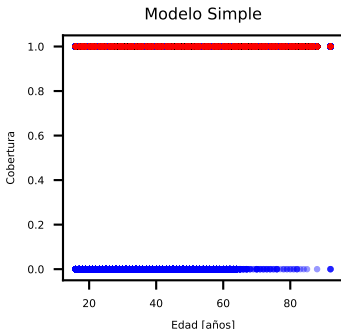
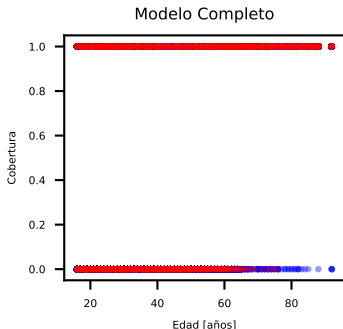
```
## Accuracy modelo simple: 0.8169774747677294
```

Vemos que los valores son similares, por lo que no pareciera ser tanto peor el modelo simple respecto del completo.

Comparación gráfica

Veamos cómo se ajustan las predicciones en ambos casos a las respuestas reales:

```
fig, axes = plt.subplots(1, 2, figsize=(4, 2)); lPreds = [y_pred, y_pred0]; tits = ["Completo", "Simple"]
for i in range(2):
    axes[i].scatter(X["age"], y, alpha=0.4, c="b", s=2)
    axes[i].scatter(X["age"], lPreds[i], alpha=0.2, c="r", marker="x", s=2)
    axes[i].set_xlabel("Edad [años]", fontsize=4); axes[i].set_ylabel("Cobertura", fontsize=4)
    axes[i].tick_params(axis="both", labelsize=4); axes[i].set_title("Modelo "+tits[i], fontsize=6);
fig.tight_layout(); plt.show()
```



Se ve que en el modelo simple ninguna persona es clasificada como "sin cobertura", por lo que directamente asignó todas las respuestas a la clase mayoritaria con cobertura, sin ningún tipo de distinción.

Comparación de métricas

Esto se ve con más claridad en las matrices de confusión:

```
CMat0=confusion_matrix(y, y_pred0)
dfCM0=pd.DataFrame({"Clas s/SM":CMat0[:,0], "Clas c/SM":CMat0[:,1]}, index=["s/SM", "c/SM"])
```

```
print("Modelo Completo:")
```

```
## Modelo Completo:
```

```
print(dfCM)
```

```
##          Clas s/SM  Clas c/SM
## s/SM           3135      10556
## c/SM           1947      59167
```

```
print("Modelo Simple:")
```

```
## Modelo Simple:
```

```
print(dfCM0)
```

```
##          Clas s/SM  Clas c/SM
## s/SM              0      13691
## c/SM              0      61114
```

Comparación de métricas

Más aún, en el `classification_report` se muestran todas las métricas:

```
print("Modelo Completo:")
```

Modelo Completo:

```
print(classification_report(y,y_pred))
```

		precision	recall	f1-score	support
##					
##	0	0.62	0.23	0.33	13691
##	1	0.85	0.97	0.90	61114
##					
##	accuracy			0.83	74805
##	macro avg	0.73	0.60	0.62	74805
##	weighted avg	0.81	0.83	0.80	74805

```
print("Modelo Simple:")
```

Modelo Simple:

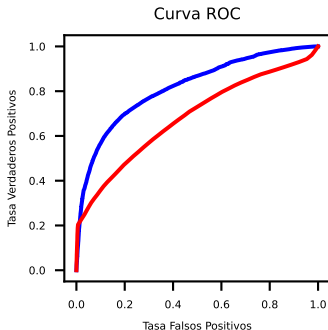
```
print(classification_report(y,y_pred0))
```

		precision	recall	f1-score	support
##					
##	0	0.00	0.00	0.00	13691
##	1	0.82	1.00	0.90	61114
##					
##	accuracy			0.82	74805
##	macro avg	0.41	0.50	0.45	74805
##	weighted avg	0.67	0.82	0.73	74805

Comparación de Curvas ROC

También es notoria la diferencia entre las curvas ROC, donde la curva del modelo simple se agrega en color rojo:

```
fig, axes = plt.subplots(1, 1, figsize=(2, 2));
fpr0, tpr0, thresholds0 = roc_curve(y, y_pred_probs0)
plt.plot(fpr, tpr, c="b"); plt.plot(fpr0, tpr0, c="r");
plt.title('Curva ROC', fontsize=6); plt.tick_params(axis="both", labelsz=4);
plt.xlabel('Tasa Falsos Positivos', fontsize=4); plt.ylabel('Tasa Verdaderos Positivos', fontsize=4);
fig.tight_layout(); plt.show()
```



Comparación de Áreas bajo la Curva ROC

La diferencia también se expresa en las áreas bajo estas curvas:

```
print("Modelo Completo:")
```

```
## Modelo Completo:
```

```
print(roc_auc_score(y, y_pred_probs))
```

```
## 0.8174189879393283
```

```
print("Modelo Simple:")
```

```
## Modelo Simple:
```

```
print(roc_auc_score(y, y_pred_probs0))
```

```
## 0.6827424900082738
```

Hemos visto una forma de modelar el vínculo entre una variable de respuesta **cualitativa binaria** con otras variables explicativas.

- **Ventajas:**
 - Al plantear un modelo, se puede **medir** la influencia de una variable sobre la variable de respuesta a través del coeficiente β .
 - Además de la respuesta, se puede estimar una probabilidad de que nuevas observaciones cumplan una de ambas condiciones, pudiendo adaptarse a la incertidumbre.
 - Evaluando la influencia de distintas variables, se pueden detectar sesgos y errores que introducen las mismas en las predicciones.
- **Desventajas:**
 - El modelo hay que armarlo a criterio y no siempre tenemos las variables necesarias para plantear un buen modelo.
 - Las ventajas dependen de que partan de un buen modelo, que como dijimos, no es fácil de conseguir.
 - Es computacionalmente más “caro” que otros modelos más simples, pero que no poseen las ventajas anteriores.

Bibliografía

Jeffrey C. Chen, Edward A. Rubin, Gary J. Cornwall - Data Science for Public Policy (Springer Series in the Data Sciences)-Springer (2021)